Krishinex user app labour and equipment booking section Optimization --

---

Labour Booking Screen – High Performance Technical Document

1. Objective

Labour Booking screen ko maximum fast aur responsive banana hai.

Target:

- Screen open hote hi blank/full-screen loader nahi dikhna chahiye.
- Existing/cached data ho to **immediately display** hona chahiye.
- Nearby labour search **MongoDB geospatial query** se honi chahiye.
- User ke location ke **10 km radius** ke andar ke labour fetch hone chahiye.
- Nearest labour pehle dikhna chahiye.
- API response lightweight hona chahiye.
- Images optimized/cached honi chahiye.
- User dusri screen par jaakar wapas aaye to **same data dobara zero se load nahi hona chahiye**.
- Fresh data background me update ho sakta hai.
- API calls unnecessary repeat nahi honi chahiye.

2. Complete Recommended Flow

P.T.O.

USER

↓

Labour Booking par click

↓

Check Cached Labour Data

↓

Data available?

├── YES → Immediately show cached labour

│

│ ↓

│ Background refresh

│

└── NO → Get current/last known location

↓

API Call

↓

Backend

↓

MongoDB

↓

10 KM Geospatial Search

↓

Nearest Labour First

↓

Lightweight Response

↓

React Native FlatList

↓

Images from Cache/CDN

3. Labour ki Location Database me Kaise Save Karni Hai

{

"_id": "LABOUR_ID",

"name": "Ram Kumar",

"location": {

"type": "Point",

"coordinates": [81.8463, 25.4358]

},

"status": "active",

"availability": true,

"locationUpdatedAt": "2026-09-02T05:30:00Z"

}

IMPORTANT

MongoDB GeoJSON me order:

[longitude, latitude]

Longitude pehle, latitude baad me.

Location unnecessarily har second update nahi karni hai.

5. MongoDB me 2dsphere Index

Location field par index create karo:
“db.labours.createIndex({

location: "2dsphere"

});”

Ye bahut important hai.

Isse MongoDB ko location-based searching efficiently karne me help milegi.

6. User ki Location Kaise Lenge?

User Labour Booking open karta hai.

App:

Last known location available?

YES

Immediately last known location use karo.

NO

Fresh GPS location obtain karo.

Example:
User Location

Latitude:

25.4358

Longitude:

81.8463

Phir backend ko bhejo.

7. Nearby Labour API

Recommended API:

GET /api/labours/nearby

Parameters:

lat=25.4358

lng=81.8463

radius=10

page=1

limit=20

Example:

GET /api/labours/nearby?lat=25.4358&lng=81.8463&radius=10&page=1&limit=20

8. MongoDB Query

MongoDB me $near use kiya ja sakta hai:

```
db.labours.find({
 status: "active",
 availability: true,
 location: {
  $near: {
   $geometry: {
    type: "Point",
    coordinates: [
     81.8463,
     25.4358
    ]
   },
   $maxDistance: 10000
  }
 }
})
.limit(20);
```

Yahan:

10000 meters = 10 KM

Aur $near nearest records ko priority de sakta hai.

9. Distance bhi Response me Dena Hai

User ko dikhana hai:

Ram Kumar

2.1 km away

Shyam

4.3 km away

Ramesh

7.8 km away

Isliye backend ko distance calculate karke response me bhejna chahiye.

Conceptual aggregation:

```
db.labours.aggregate([
 {
  $geoNear: {
   near: {
    type: "Point",
    coordinates: [81.8463, 25.4358]
   },
   key: "location",
   distanceField: "distanceMeters",
   maxDistance: 10000,
   spherical: true,
   query: {
    status: "active",
    availability: true
   }
  }
 },
 {
  $limit: 20
 }
]);
```

Phir:

distanceMeters / 1000

se kilometre calculate kar sakte ho.

10. API Response Lightweight Rakho

Sabse important optimization yahi hai.

Labour list API me **full profile mat bhejo**.

❌ **Galat**

```
{
 "name": "...",
 "phone": "...",
 "address": "...",
 "documents": "...",
 "bankDetails": "...",
 "allImages": "...",
 "experience": "...",
 "fullProfile": "..."
}
```

✅ Sahi

JSON Responce

```
{
 "data": [
  {
   "id": "101",
   "name": "Ram Kumar",
   "distance": 2.1,
   "rating": 4.6,
   "profileImage": "thumbnail-url",
   "available": true
  }
 ],

 "page": 1,
 "limit": 20,
 "hasMore": true
}
```

List screen ko sirf list ke liye required data mile.

11. Full Labour Profile Kab Load Hogi?

User labour card par click kare:

Ram Kumar

↓

Labour Profile Screen

↓

GET /api/labours/101

↓

Full profile

Yaani:

List API

↓

Small data

Profile API

↓

Full data

Isse initial Labour Booking screen bahut lightweight rahegi.

12. Pagination

Ek baar me 100 labour mat bhejo.

Example:

First API:

20 labour

User scrolls:

Next 20

Again scroll:

Next 20

API:

?page=1&limit=20

Next:

?page=2&limit=20

Isse initial loading fast hogi.

13. React Native me Screen Reopen Hone Par Kya Karna Hai?

Ye tumhare current issue ka sabse important part hai.

Abhi shayad:

Labour Booking

↓

API

↓

Data

Other Screen

↓

Back to Labour Booking

↓

API AGAIN

↓

GPS AGAIN

↓

Loading AGAIN

Ye nahi hona chahiye.

Recommended:

Labour Booking

↓

API

↓

Data

↓

Cache

Ab user dusri screen par gaya:

Labour Booking

↓

Other Screen

↓

Back

↓

Cached Labour Data

↓

IMMEDIATELY SHOW

Phir background me:

Fresh API

↓

Latest data

↓

Cache update

↓

UI update

14. Cache Strategy

React Native application me server-state caching ke liye **TanStack Query / React Query** jaisi library use ki ja sakti hai.

Concept:

Query Key: ["nearbyLabours", latitude, longitude]

Data cache me rahega.

Example configuration concept:

```
useQuery({
  queryKey: ["nearbyLabours", latitude, longitude],
  queryFn: fetchNearbyLabours,
  staleTime: 60 * 1000,
  gcTime: 5 * 60 * 1000
});
```

Meaning:

60 sec:

Data fresh maana ja sakta hai.

5 min:

Cache memory me retain ho sakta hai.

Exact timing labour availability requirements ke according set karni hai.

15. Background Refresh

User ko wait mat karao.

Example:

Cached Data

↓

Immediately Show

↓

Background API Call

↓

Latest Data

↓

Update Screen

User ko:

Loading...

par 3–5 second nahi rukna padega.

16. Location Cache

Location ko bhi cache karna hai.

Example:

lastLatitude

lastLongitude

locationTimestamp

Agar location bahut recent hai:

Use cached location

Agar purani hai:

Get fresh location

18. Images Optimization

Bhai **bahut baar actual slow loading images ki wajah se hoti hai.**

Labour list me:

❌ **Full image**

2 MB
3 MB
5 MB

mat load karo.

✅ **Thumbnail**

Example:

50 KB – 150 KB

range ka optimized thumbnail use karo, actual dimensions/content ke according.

Profile open hone par:

Full-quality image

load ho sakti hai.

Image ko CDN/object storage + caching ke through serve karna preferable hai.

19. React Native FlatList

Labour list ke liye:

FlatList

use karo.

ScrollView me hundreds of labour cards ek saath render nahi karne hain.

Concept:

```
<FlatList
 data={labours}
 renderItem={renderLabour}
 keyExtractor={(item) => item.id}
 onEndReached={loadNextPage}
 onEndReachedThreshold={0.5}
/>
```

Further optimization project ke actual card complexity ke according ki ja sakti hai.

20. API Calling Rules

First time:

Location

↓

Nearby Labour API

↓

Cache

↓

Show

Same screen par wapas:

Cache

↓

Show Immediately

↓

Background Refresh

Location changed:

New Location

↓

New API

↓

Update Cache

Profile open:

Profile API

Next page:

Pagination API

21. Duplicate API Calls Avoid Karna

Ye bahut important hai.

Aisa nahi hona chahiye:

Component Mount

↓ API

useEffect

↓ API

Location Callback

↓ API

Screen Focus

↓

API

Result:

Same API × 3/4

Isko control karna hai.

Ek hi query manager/cache layer honi chahiye.

22. API Request Deduplication

Agar same request already running hai:

Request #1

aur user ne same screen dobara open kar diya:

Request #2 ❌

nahi bhejna.

Existing request ka result use karo.

React Query/TanStack Query jaise tools is pattern ko manage karne me help kar sakte hain.

23. Backend Response Time Measure Karna

Actual timing measure karo:

GPS:

___ ms

API:

___ ms

MongoDB:

___ ms

Response:

___ KB

Image:

___ ms

React Native render:

___ ms

Tab pata chalega bottleneck kahan hai.

24. MongoDB Query Performance Check

Developer ko:

Js Quiry explain("executionStats")

se query inspect karni chahiye.

Example concept:

```
db.labours.explain("executionStats").aggregate([
 {
  $geoNear: {
    near: {
     type: "Point",
     coordinates: [81.8463, 25.4358]
    },
    key: "location",
    distanceField: "distanceMeters",
    maxDistance: 10000,
    spherical: true
  }
 },
 {
  $limit: 20
 }
]);
```

Check:

executionTimeMillis

totalDocsExamined

totalKeysExamined

25. 1 Second Target

Goal:

User clicks Labour Booking

↓

Existing cache?

↓

YES

↓

Instant display

Ye approach **1 second se bhi kam perceived loading** de sakti hai, kyunki network ka wait nahi karna padega.

First-ever visit par:

GPS

↓

API

↓

MongoDB

↓

Response

↓

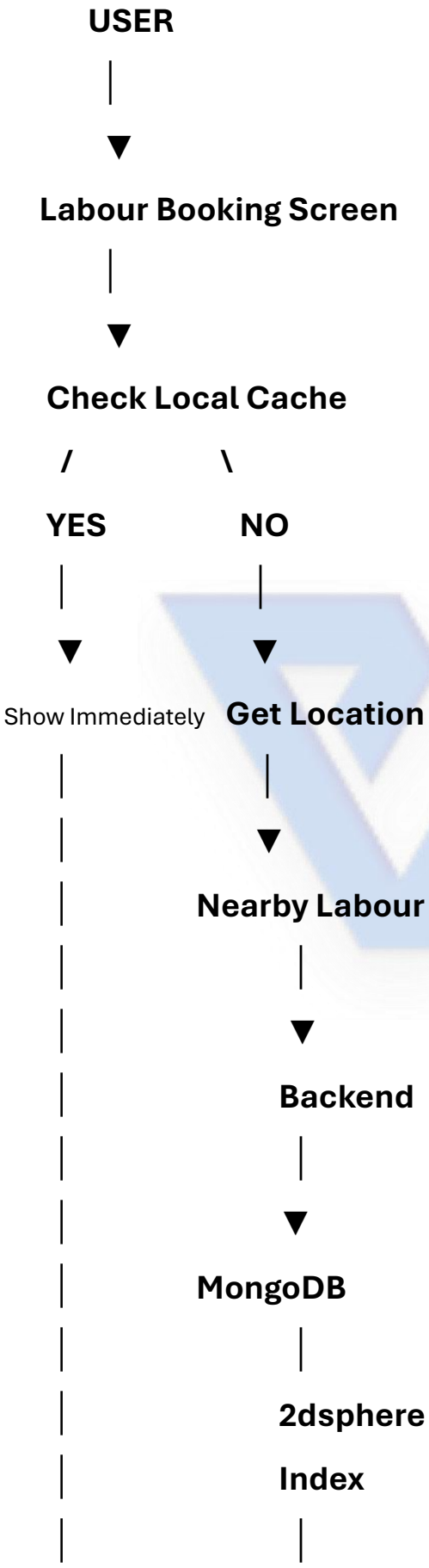
Images

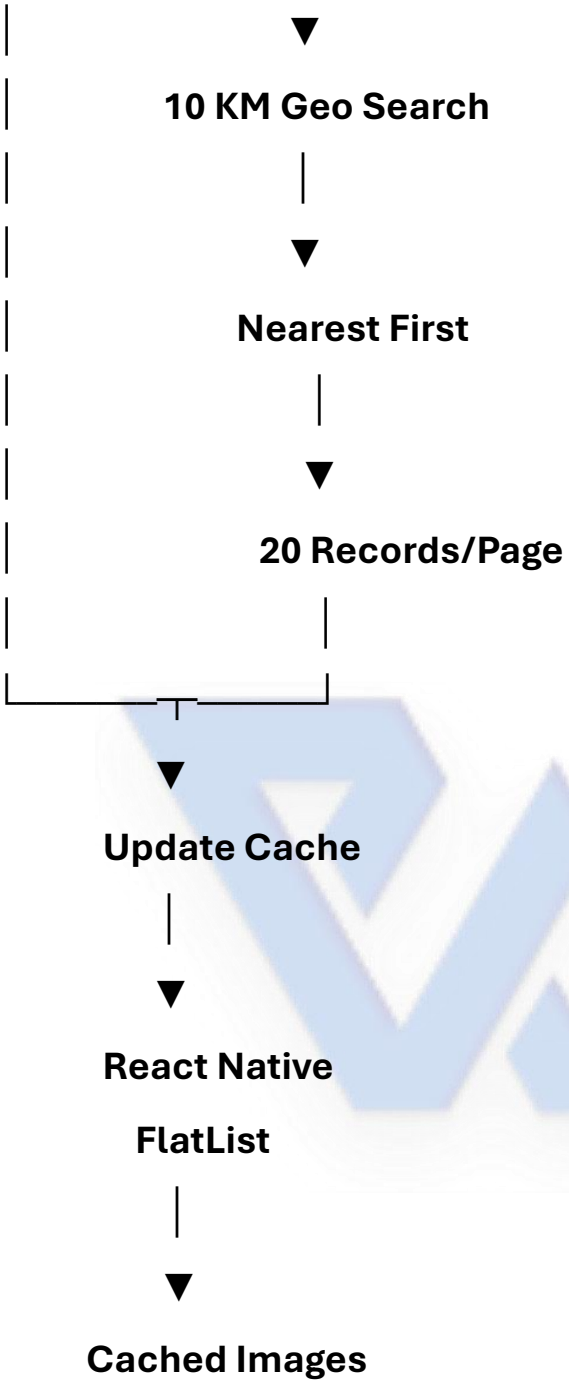
hone ki wajah se exact 1 second guarantee nahi ki ja sakti.

Isliye **first load + cached subsequent load** dono optimize karna hai.

26. Ideal Final Architecture

P.T.O.


USER
Labour Booking Screen
Check Local Cache
YES
NO
Show Immediately
Get Location
Nearby Labour API
Backend
MongoDB
2dsphere
Index


10 KM Geo Search
Nearest First
20 Records/Page
Update Cache
React Native
FlatList
Cached Images

27. Developer ke liye Final Checklist

☐ Labour location GeoJSON Point me store karo

☐ coordinates [longitude, latitude] rakho

☐ location field par 2dsphere index lagao

☐ MongoDB $geoNear / appropriate geospatial query use karo

☐ 10 KM radius database level par filter karo

☐ Active/available labour query level par filter karo

☐ Nearest labour first return karo

☐ First API response maximum required fields hi bheje

☐ Pagination implement karo

☐ Full profile list API me mat bhejo

☐ Profile details separate API se lao

☐ Profile thumbnails use karo

☐ Images cache/CDN se serve karo

☐ React Native FlatList use karo

☐ Location ko unnecessarily baar-baar fetch mat karo

☐ Same API duplicate call mat hone do

☐ Labour list ko cache karo

☐ Screen reopen par cached data immediately show karo

☐ Background me fresh data fetch karo

☐ Location significantly change hone par new search karo

☐ MongoDB query ko explain("executionStats") se test karo

☐ API response time log karo

☐ Database query time log karo

☐ Response payload size check karo

☐ Release APK/AAB me performance test karo

Sabse important conclusion:

Best Approch

Cache First → Instant UI → Background Refresh → MongoDB Geospatial Search → Lightweight API → Pagination → Cached Images

Is architecture se user ko har baar **GPS → API → MongoDB → profiles → images** wala poora process wait karke nahi dekhna padega.

This Document process get only 7 to 8 hour by using antigravity using hight speed internet.